Problem, Team, and Overview of Experiments

When tickets for a popular concert go on sale, thousands of users simultaneously try to book the same seats and the system must handle that spike without overselling, crashing, or leaving users waiting. Real platforms have faced public failures in exactly this scenario, and the root causes are almost always architectural. This project builds a simulated concert ticket booking backend to study how specific design decisions affect system behavior under load, isolating one variable at a time across five experiments. The team consists of Abhishek Raghuraman, Bhargav Pamidighantam, and Megan Heinhold. AI was used in a supporting role — Claude helped the team understand design tradeoffs, assisted with documentation and report writing, and generated charts from experiment data. The team made all architectural decisions, experiment design, and result interpretation.

Project Plan and Recent Progress

So far, the team has completed the infrastructure setup and the first two experiments. The full platform was built from scratch — three Go microservices (inventory, booking, queue), modular Terraform covering VPC, ALB, ECS Fargate, RDS MySQL, DynamoDB, ECR, CloudWatch, and autoscaling, plus deploy, cleanup, and smoke test scripts. The platform was verified working on both MySQL and DynamoDB backends. Experiment 1 tested all three concurrency control modes against both database backends with 1000 users. Experiment 2 compared direct versus queue buffered booking under a 500-user flash sale load. Results from both experiments are included in this report.

The remaining three experiments covering autoscaling behavior, multi event isolation, and fairness are the next priority. End to end integration testing across all experiments, the final report, and the dashboard UI are also planned. AI was used throughout to accelerate infrastructure development and documentation, which gave the team more time to focus on experiment design and result analysis rather than boilerplate setup work.

Objectives

The short-term goal is to complete all five experiments across both database backends and produce evidence-based recommendations for each architectural tradeoff. The long-term vision is to turn this into a reusable benchmarking tool for distributed ticketing systems. Teams building high demand booking systems face the same questions we are studying - which locking strategy fits their consistency requirements, whether a virtual queue protects downstream services, and how to configure autoscaling for spike traffic. A configurable reproducible platform that answers these questions with real data would be useful beyond this course.

Related Work

The Parnas modularity paper shaped our decision to split the platform into three services that each hide a design decision from the others, allowing the team to work independently without blocking each other. The async messaging patterns from Homework 7 directly informed the queue service — the core idea of decoupling arrival rate from processing rate is the same, and our results confirm the same finding. The horizontal scaling work from Homework 6 informs Experiment 3, applying the principle of adding instances rather than optimizing code for spike load. Outside the course, Kleppmann's Designing Data Intensive Applications provides the theoretical grounding for the MySQL versus DynamoDB comparison — MySQL's ACID guarantees make row locking natural and efficient, while DynamoDB's eventual consistency model requires conditional writes to simulate locking, which our results show comes at a significant latency cost under contention.

Methodology

The platform has three Go microservices behind a single ALB with path-based routing. The inventory service manages events and seats. The booking service is the main subject of all experiments and implements all three concurrency modes in the same binary — no lock, optimistic locking with version based conditional writes, and pessimistic locking using SELECT FOR UPDATE in MySQL or a conditional in progress flag in DynamoDB. The queue service manages the virtual waiting room with a configurable admission rate and fairness mode. All three services support MySQL and DynamoDB as backends, switchable at runtime via an environment variable with no code changes.

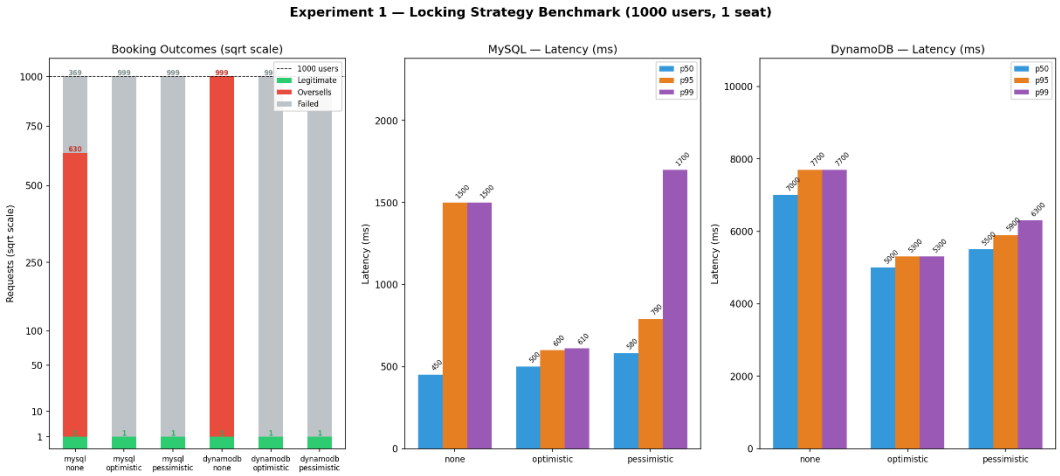
Load testing uses Locust with one file per experiment, and infrastructure changes between runs are made through Terraform variable overrides. Observability is through CloudWatch for CPU and memory per service, Locust for response times and failure rates, and a custom metrics endpoint on the booking service that reports oversell counts in real time. The pathological worst case is no lock mode on DynamoDB under high contention. The best-case baseline is pessimistic locking on MySQL with the queue enabled.

Preliminary Results

Experiment 1: Concurrency Control Under Flash Sale Load

1000 concurrent users all targeted the same single seat across all six combinations of lock mode and database backend. On MySQL, no lock produced 630 oversells out of 631 bookings at an average of 538ms — the fastest but completely incorrect. Optimistic locking brought oversells to zero at 453ms average, and pessimistic locking also produced 0 oversells at 539ms with a p99 of 1700ms reflecting the tail of requests waiting in the lock

queue. DynamoDB told a very different story — no lock produced 999 oversells out of 1000 bookings at an average of 6537ms, more than 12 times slower than MySQL No lock. Even with locking enabled, DynamoDB averaged 3946ms and 4803ms for optimistic and pessimistic respectively, around 9 times slower than the equivalent MySQL modes.

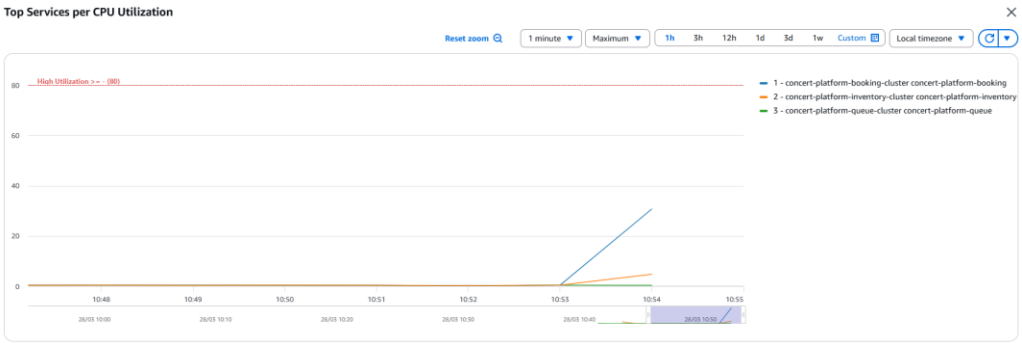


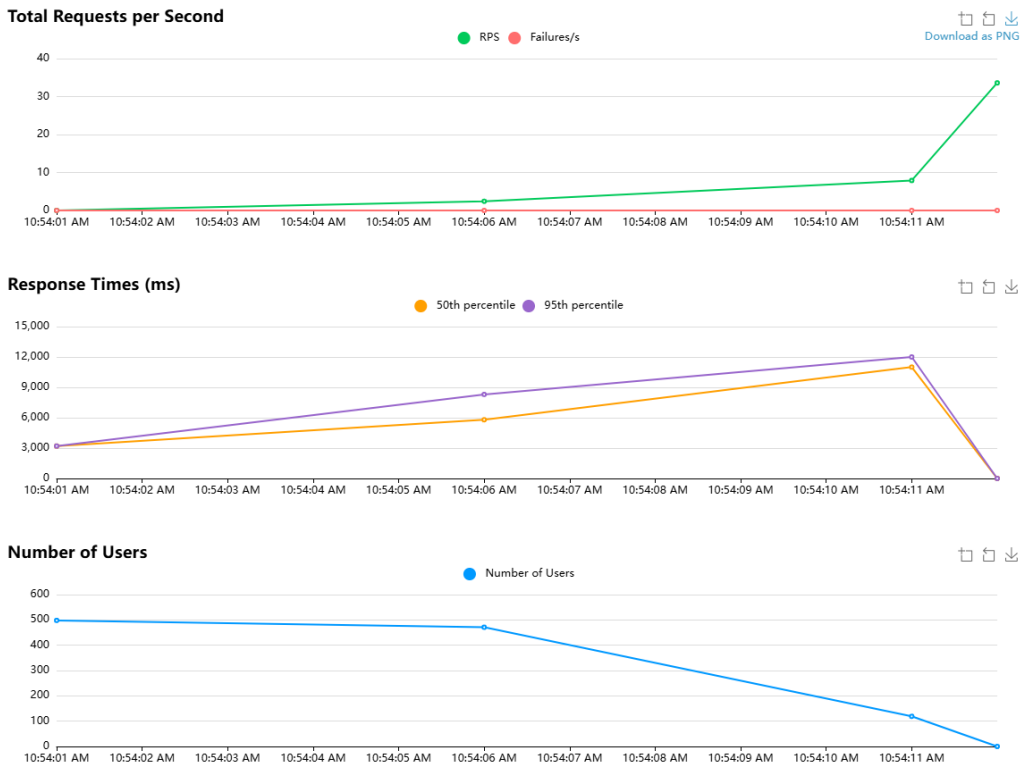
The key finding is that MySQL's native row locking is far more efficient than DynamoDB's conditional write simulation under high contention. Both databases achieve the same correctness outcome when locking is enabled, but MySQL does it at a fraction of the latency. For flash sale scenarios where many users compete for the same seat simultaneously, MySQL pessimistic locking provides the best combination of correctness and predictable response times.

Experiment 2: Virtual Queue as a Demand Buffer

500 users made booking requests at a ramp up of 200 users per second. Tests compared direct booking versus queue buffered booking on both backends. All tests used pessimistic locking, fairness mode allow multiple, and admission rate of 10 per second.

Test 1: DynamoDB Direct Booking

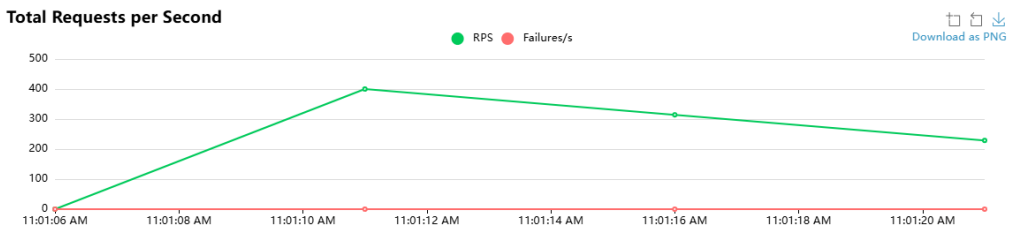


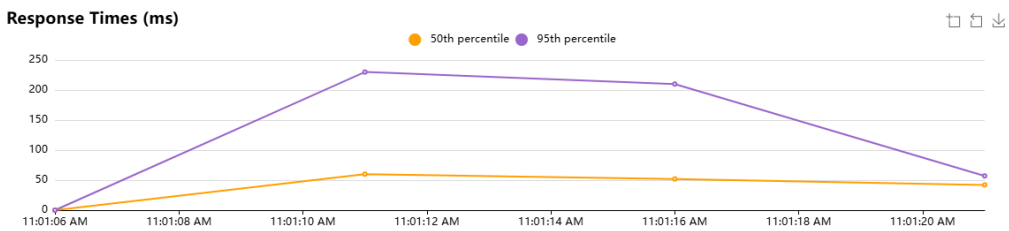


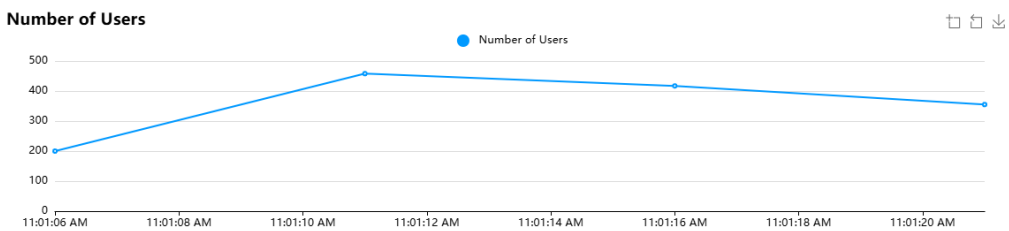
100% success rate but response times climbed steadily throughout the test, reaching around 9000ms at the 50th percentile and 11000ms at the 95th percentile by the end. ECS CPU for the booking service peaked at around 30% and memory at 22%, with a sharp spike visible in CloudWatch as all 500 users hit the service simultaneously.

Test 2: DynamoDB with Queue

```
PS C:\Users\Megan\Documents\CS6650FinalProject\CS6650-Flash-Sale> $response1 | Format-List *

queues : {@{event_id=evt-001; queue_depth=472; admission_rate_hz=10; total_admitted=529; total_waiting=472;
         fairness_mode=allow_multiple}}


PS C:\Users\Megan\Documents\CS6650FinalProject\CS6650-Flash-Sale> $response2 | Format-List *

queues : {@{event_id=evt-001; queue_depth=422; admission_rate_hz=10; total_admitted=579; total_waiting=422;
         fairness_mode=allow_multiple}}


PS C:\Users\Megan\Documents\CS6650FinalProject\CS6650-Flash-Sale> $response3 | Format-List *

queues : {@{event_id=evt-001; queue_depth=286; admission_rate_hz=10; total_admitted=715; total_waiting=286;
         fairness_mode=allow_multiple}}


                                                                  $response4 | Format-List *
PS C:\Users\Megan\Documents\CS6650FinalProject\CS6650-Flash-Sale> $response3 | Format-List *

queues : {@{event_id=evt-001; queue_depth=174; admission_rate_hz=10; total_admitted=827; total_waiting=174;
         fairness_mode=allow_multiple}}
```



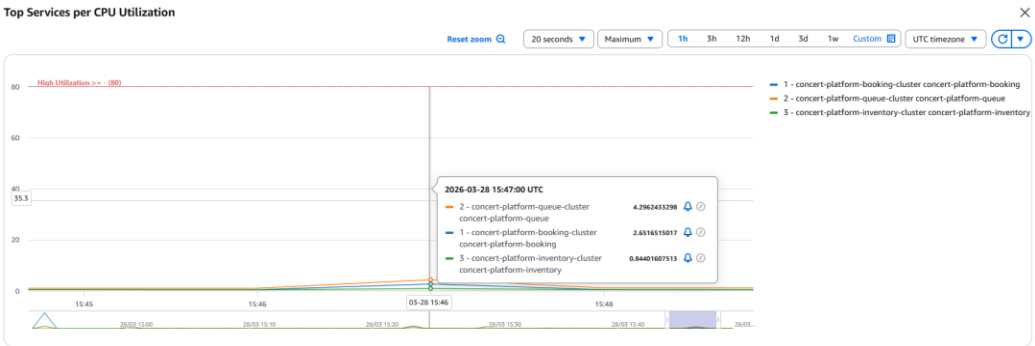




Response times dropped to around 50ms at the 50th percentile and 225ms at the 95th percentile — a 98% reduction compared to direct booking. Success rate stayed at 100%. Queue depth peaked at 472 with 529 already admitted and continued draining steadily. ECS CPU and memory stayed nearly flat across all three services, peaking under 5%.

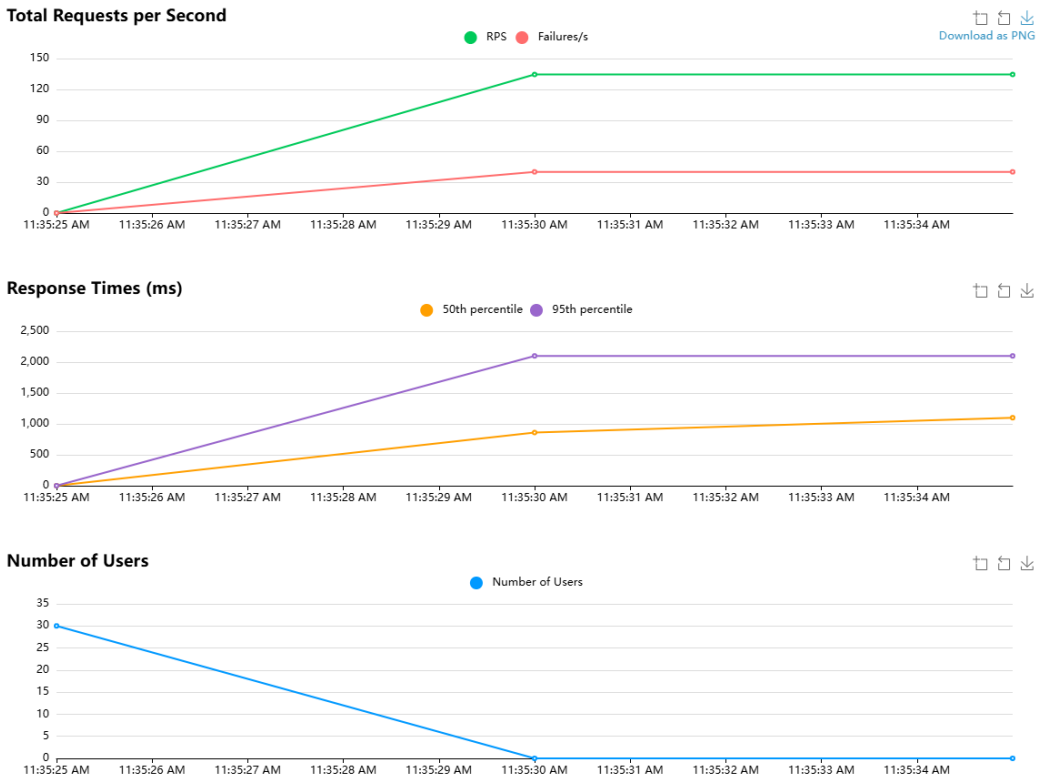
Test 3: MySQL with Queue

```
PS C:\Users\Megan\Documents\CS6650FinalProject\CS6650-Flash-Sale> $response1 | Format-List *

queues : {@{event_id=evt-001; queue_depth=473; admission_rate_hz=10; total_admitted=1029; total_waiting=473;
         fairness_mode=allow_multiple}}


PS C:\Users\Megan\Documents\CS6650FinalProject\CS6650-Flash-Sale> $response2 | Format-List *

queues : {@{event_id=evt-001; queue_depth=410; admission_rate_hz=10; total_admitted=1092; total_waiting=410;
         fairness_mode=allow_multiple}}


PS C:\Users\Megan\Documents\CS6650FinalProject\CS6650-Flash-Sale> $response3 | Format-List *

queues : {@{event_id=evt-001; queue_depth=299; admission_rate_hz=10; total_admitted=1203; total_waiting=299;
         fairness_mode=allow_multiple}}


PS C:\Users\Megan\Documents\CS6650FinalProject\CS6650-Flash-Sale> $response4 | Format-List *

queues : {@{event_id=evt-001; queue_depth=226; admission_rate_hz=10; total_admitted=1276; total_waiting=226;
         fairness_mode=allow_multiple}}
```



Failure rate dropped from 14% to 0.1% (7 out of 3910 responses). Average response time improved to 73ms and p95 to 230ms. Queue depth peaked at 473 with 1029 admitted and steadily draining. ECS CPU peaked at 4.3% for the queue service and 2.6% for booking — lower than direct booking despite processing significantly more total requests.

Test 4: MySQL Direct Booking



14% failure rate with 128 out of 500 requests returning 409 errors from lock contention building up faster than the system could resolve it. Average response time was 902ms and p95 was 2000ms.

The most important finding from Experiment 2 is that the queue does not just improve response times. It improves correctness. MySQL direct booking had a 14% failure rate from lock contention. Adding the queue in front dropped that to near zero because requests arrive at the booking service at a controlled rate rather than all at once, preventing contention from building up. The queue is acting as a correctness mechanism, not just a performance one.

Impact

The results from this project have practical value for anyone designing a high-demand booking system. DynamoDB locking being 9 to 12 times slower than MySQL under contention is not obvious from either database's documentation, and the finding that a queue improves correctness and not just performance — dropping MySQL's failure rate from 14% to 0.1% challenges the common assumption that queues are purely a throughput mechanism. Other students can deploy and test this platform with a single script since all experiment parameters are runtime configurable, and the three remaining experiments will add further evidence to the dataset.